Creacion de pedidos de una linea y multilinea.

Script inicial:

```
CREATE SCHEMA tienda;
SET search_path TO tienda;
CREATE TABLE clientes (
cliente_id SERIAL PRIMARY KEY,
nombre TEXT NOT NULL,
email TEXT UNIQUE NOT NULL,
fecha_alta DATE NOT NULL DEFAULT CURRENT_DATE
);
CREATE TABLE productos (
producto_id SERIAL PRIMARY KEY,
nombre TEXT NOT NULL,
precio NUMERIC(10,2) NOT NULL,
stock INTEGER NOT NULL DEFAULT 0,
activo BOOLEAN NOT NULL DEFAULT TRUE
);
CREATE TABLE pedidos (
pedido_id SERIAL PRIMARY KEY,
cliente_id INTEGER NOT NULL REFERENCES clientes(cliente_id),
fecha_pedido TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
estado TEXT NOT NULL DEFAULT 'pendiente'
);
CREATE TABLE lineas_pedido (
linea_id SERIAL PRIMARY KEY,
pedido_id INTEGER NOT NULL REFERENCES pedidos(pedido_id),
producto_id INTEGER NOT NULL REFERENCES productos(producto_id),
cantidad INTEGER NOT NULL,
precio_unit NUMERIC(10,2) NOT NULL
);
CREATE TABLE pagos (
pago_id SERIAL PRIMARY KEY,
pedido_id INTEGER NOT NULL REFERENCES pedidos(pedido_id),
fecha_pago TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
importe NUMERIC(10,2) NOT NULL,
metodo TEXT NOT NULL
);
CREATE TABLE empleados (
id SERIAL PRIMARY KEY,
nombre TEXT NOT NULL,
salario NUMERIC(10,2) NOT NULL,
departamento TEXT,
fecha_contratacion DATE DEFAULT CURRENT_DATE
);
INSERT INTO empleados (nombre, salario, departamento) VALUES
('Ana Torres', 2800.00, 'Ventas'),
('Luis Martínez', 4500.00, 'IT'),
('Carla Gómez', 5200.00, 'IT'),
('Dmitry Komarov', 3100.00, 'Contabilidad'),
('James Smith', 7000.00, 'Dirección'),
('Mary Sexton', 3900.00, 'Marketing'),
('Agnes Miller', 6100.00, 'IT'),
('Chun Yen', 2950.00, 'Ventas');
INSERT INTO clientes (nombre, email) VALUES
('Juan Pérez', 'juan@example.com'),
('María López', 'maria@example.com');
INSERT INTO productos (nombre, precio, stock) VALUES
```

('Teclado mecánico', 79.90, 50),
('Ratón inalámbrico', 39.50, 100),
('Monitor 27"', 249.00, 20);
INSERT INTO pedidos (cliente_id, estado) VALUES
(1, 'pendiente'),
(2, 'pendiente');
INSERT INTO lineas_pedido (pedido_id, producto_id, cantidad, precio_unit) VALUES
(1, 1, 1, 79.90),
(1, 2, 2, 39.50),
(2, 3, 1, 249.00);
INSERT INTO pagos (pedido_id, importe, metodo) VALUES
(1, 158.90, 'tarjeta'),
(2, 249.00, 'transferencia');

Procedure para Crear un pedido, crear su línea y reducir stock de manera atómica.

```
CREATE OR REPLACE PROCEDURE tienda.sp_crear_linea_pedido(
    IN p_cliente_id integer,
    IN p_producto_id integer,
    IN p_cantidad integer,
    OUT v_pedido_id integer,
    OUT v_linea_id integer,
    OUT v_total_linea numeric(10,2)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_precio_unit numeric(10,2);
    v_stock integer;
BEGIN
    IF p_cantidad <= 0 THEN
        RAISE EXCEPTION 'La cantidad debe ser mayor que cero';
    END IF;

    -- FOR UPDATE evita que dos ventas concurrentes descuenten el mismo stock.
    SELECT precio, stock
      INTO STRICT v_precio_unit, v_stock
      FROM tienda.productos
     WHERE producto_id = p_producto_id
       AND activo
     FOR UPDATE;

    IF v_stock < p_cantidad THEN
        RAISE EXCEPTION 'Stock insuficiente para producto %: disponible %,
solicitado %',
            p_producto_id, v_stock, p_cantidad;
    END IF;

    INSERT INTO tienda.pedidos (cliente_id, estado)
    VALUES (p_cliente_id, 'pendiente')
    RETURNING pedido_id INTO v_pedido_id;

    INSERT INTO tienda.lineas_pedido (pedido_id, producto_id, cantidad,
precio_unit)
    VALUES (v_pedido_id, p_producto_id, p_cantidad, v_precio_unit)
    RETURNING linea_id INTO v_linea_id;
```

```
    UPDATE tienda.productos
       SET stock = stock - p_cantidad
     WHERE producto_id = p_producto_id;

    v_total_linea := round(p_cantidad * v_precio_unit, 2);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION 'Producto activo no encontrado: %', p_producto_id;
END;
$$;


-- Los lugares para OUT se pasan como NULL en CALL.
CALL tienda.sp_crear_linea_pedido(1, 1, 2, NULL, NULL, NULL);
```

Resultado esperado de `CALL`: una fila de salida con un nuevo `v_pedido_id`, el nuevo `v_linea_id` y `v_total_linea = 159.80`. Además, el stock del producto 1 disminuye en 2.

procedimiento convertido a función

Aquí tienes la versión como `FUNCTION`. La lógica interna es **idéntica**; solo cambia la firma y cómo se declaran los valores de retorno (en vez de `OUT`, se usa `RETURNS TABLE`).

sql

```
CREATE OR REPLACE FUNCTION tienda.fn_crear_linea_pedido(
    p_cliente_id  integer,
    p_producto_id integer,
    p_cantidad    integer
)
RETURNS TABLE(
    v_pedido_id   integer,
    v_linea_id    integer,
    v_total_linea numeric(10,2)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_precio_unit numeric(10,2);
    v_stock       integer;
BEGIN
    IF p_cantidad <= 0 THEN
        RAISE EXCEPTION 'La cantidad debe ser mayor que cero';
    END IF;

    -- FOR UPDATE evita que dos ventas concurrentes descuenten el mismo stock.
    SELECT precio, stock
      INTO STRICT v_precio_unit, v_stock
      FROM tienda.productos
     WHERE producto_id = p_producto_id
       AND activo
     FOR UPDATE;

    IF v_stock < p_cantidad THEN
        RAISE EXCEPTION 'Stock insuficiente para producto %: disponible %,
solicitado %',
            p_producto_id, v_stock, p_cantidad;
    END IF;
```

```
    INSERT INTO tienda.pedidos (cliente_id, estado)
    VALUES (p_cliente_id, 'pendiente')
    RETURNING pedido_id INTO v_pedido_id;

    INSERT INTO tienda.lineas_pedido (pedido_id, producto_id, cantidad,
precio_unit)
    VALUES (v_pedido_id, p_producto_id, p_cantidad, v_precio_unit)
    RETURNING linea_id INTO v_linea_id;

    UPDATE tienda.productos
       SET stock = stock - p_cantidad
     WHERE producto_id = p_producto_id;

    v_total_linea := round(p_cantidad * v_precio_unit, 2);

    RETURN NEXT;   -- emite la fila con los valores actuales de las variables de
salida
    RETURN;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION 'Producto activo no encontrado: %', p_producto_id;
END;
$$;
```

Qué cambió respecto al procedimiento

Aspecto	Procedimiento original	Función nueva
Cabecera	`PROCEDURE ... (IN ..., OUT ...)`	`FUNCTION ... (IN ...) RETURNS TABLE(...)`
Parámetros de salida	`OUT v_pedido_id`, `OUT v_linea_id`, `OUT v_total_linea`	Columnas de `RETURNS TABLE(...)`
Variables de salida	Declaradas automáticamente por los `OUT`	Declaradas automáticamente por `RETURNS TABLE` (mismos nombres)
Emisión del resultado	Implícita al terminar el procedimiento	Explícita con `RETURN NEXT` + `RETURN`
Invocación	`CALL ...`	`SELECT * FROM ...`
Uso en `SELECT`/`JOIN`/`WHERE`	No	Sí

Detalles importantes

1. Los nombres de las columnas de `RETURNS TABLE` son variables locales

En PL/pgSQL, las columnas declaradas en `RETURNS TABLE` **se comportan como variables** dentro de la función. Por eso puedes hacer:

sql

```
RETURNING pedido_id INTO v_pedido_id;
```

sin declarar `v_pedido_id` en el bloque `DECLARE`: ya existe como variable de salida. Lo mismo con `v_linea_id` y `v_total_linea`.

Cuidado con la ambigüedad de nombres: si dentro de la función haces un `SELECT` que mencione una columna llamada igual que una variable de `RETURNS TABLE` (por ejemplo, si `tienda.pedidos` tuviera una columna `v_pedido_id`), PostgreSQL podría confundirse. En tu caso no ocurre porque los nombres `v_*` no coinciden con columnas reales. Si alguna vez coinciden, califica la columna con el alias de la tabla (`p.pedido_id`) o renombra la variable.

2. `RETURN NEXT` vs `RETURN QUERY`

•**`RETURN NEXT;`** (sin argumentos) emite **una fila** con los valores actuales de las variables de salida. Es lo que usamos aquí: una sola fila.

•**`RETURN QUERY ...`** emite **todas las filas** de una consulta. No es lo que queremos en este caso, porque la función hace inserciones y devuelve una sola fila de resultado.

3. `RETURN;` final

Después del `RETURN NEXT`, el `RETURN;` sin valor marca el fin de la función. En la práctica, al llegar al `END` también terminaría, pero ponerlo explícito es más claro y evita que el `EXCEPTION` se dispare por accidente si añades código después.

4. El bloque `EXCEPTION` sigue funcionando igual

`NO_DATA_FOUND` lo lanza `INTO STRICT` cuando el `SELECT ... FOR UPDATE` no encuentra fila (producto inexistente o inactivo). Igual que en el procedimiento.

5. `STABLE` no aplica aquí

No pongas `STABLE` (ni `IMMUTABLE`) en esta función: **modifica datos** (`INSERT`, `UPDATE`). Debe ser `VOLATILE`, que es el valor por defecto. Si la marcas `STABLE`, PostgreSQL podría cachear el resultado o rechazar la escritura.

Cómo llamarla

sql

```
-- Devuelve una fila con las tres columnas
SELECT * FROM tienda.fn_crear_linea_pedido(1, 1, 2);
```

Resultado:

text

```
 v_pedido_id | v_linea_id | v_total_linea
-------------+------------+---------------
           1 |          1 |         25.00
```

Y ahora **sí** puedes usarla en contextos donde un procedimiento no se puede, por ejemplo:

sql

```
-- Insertar el resultado en otra tabla
INSERT INTO tienda.auditoria (pedido_id, linea_id, total)
SELECT * FROM tienda.fn_crear_linea_pedido(1, 1, 2);

-- Usarla en un JOIN (poco común pero posible)
SELECT p.cliente_id, f.v_total_linea
  FROM tienda.clientes p
  CROSS JOIN LATERAL tienda.fn_crear_linea_pedido(1, 1, 2) f;
```

Nota sobre transacciones

Al ser `FUNCTION`, **no puede hacer `COMMIT`/`ROLLBACK`** internamente. Corre dentro de la transacción del llamador: si el `SELECT`/`INSERT` que la invoca falla después, todo se revierte junto. Para este caso (crear pedido + línea + descontar stock de forma atómica) eso es exactamente lo que quieres. Si algún día necesitas confirmar por etapas dentro de la rutina, tendrías que volver al `PROCEDURE`.

Procedimiento para crear un pedido con varias líneas

La idea: recibir el cliente y un **array de líneas** (producto + cantidad), y procesarlas todas dentro del **mismo pedido** y la **misma transacción**. Si una línea falla (stock insuficiente, producto inactivo, cantidad inválida), **todo el pedido se revierte** gracias al bloque `EXCEPTION`.

Diseño de los parámetros

Para pasar varias líneas uso un **array de un tipo compuesto** o, más simple y portable, un **array de `integer`** con cantidades en posiciones pares... pero eso es feo. Mejor: dos arrays paralelos (`p_producto_ids`, `p_cantidades`) o un tipo compuesto. Voy con **dos arrays paralelos** porque es lo más simple de llamar desde SQL y desde PL/pgSQL sin crear tipos.

Alternativa más limpia si puedes crear tipos: `CREATE TYPE tienda.linea_input AS (producto_id integer, cantidad integer);` y recibir `p_lineas tienda.linea_input[]`. Lo dejo comentado al final.

El procedimiento

sql

```
CREATE OR REPLACE PROCEDURE tienda.sp_crear_pedido_multilinea(
    IN  p_cliente_id    integer,
    IN  p_producto_ids  integer[],
    IN  p_cantidades    integer[],
    OUT v_pedido_id     integer,
    OUT v_total_pedido  numeric(12,2),
    OUT v_num_lineas    integer
)
```

```
LANGUAGE plpgsql
AS $$
DECLARE
    v_precio_unit numeric(10,2);
    v_stock       integer;
    v_linea_id    integer;
    v_total_linea numeric(10,2);
    v_i           integer;
BEGIN
    -- 1) Validaciones de entrada
    IF p_producto_ids IS NULL OR p_cantidades IS NULL
       OR array_length(p_producto_ids, 1) IS NULL
       OR array_length(p_cantidades, 1) IS NULL THEN
        RAISE EXCEPTION 'Debe indicar al menos una línea (producto y cantidad)';
    END IF;

    IF array_length(p_producto_ids, 1) <> array_length(p_cantidades, 1) THEN
        RAISE EXCEPTION 'Los arrays de productos (%) y cantidades (%) deben
tener la misma longitud',
            array_length(p_producto_ids, 1), array_length(p_cantidades, 1);
    END IF;

    -- 2) Crear la cabecera del pedido UNA sola vez
    INSERT INTO tienda.pedidos (cliente_id, estado)
    VALUES (p_cliente_id, 'pendiente')
    RETURNING pedido_id INTO v_pedido_id;

    v_total_pedido := 0;
    v_num_lineas   := 0;

    -- 3) Recorrer las líneas
    FOR v_i IN 1 .. array_length(p_producto_ids, 1) LOOP
        IF p_cantidades[v_i] IS NULL OR p_cantidades[v_i] <= 0 THEN
            RAISE EXCEPTION 'Línea %: la cantidad debe ser mayor que cero
(recibido %)',
                v_i, p_cantidades[v_i];
        END IF;

        -- Bloqueo de la fila del producto para evitar sobreventa concurrente
        SELECT precio, stock
          INTO STRICT v_precio_unit, v_stock
          FROM tienda.productos
         WHERE producto_id = p_producto_ids[v_i]
           AND activo
         FOR UPDATE;

        IF v_stock < p_cantidades[v_i] THEN
            RAISE EXCEPTION 'Línea %: stock insuficiente para producto %
(disponible %, solicitado %)',
                v_i, p_producto_ids[v_i], v_stock, p_cantidades[v_i];
        END IF;

        INSERT INTO tienda.lineas_pedido (pedido_id, producto_id, cantidad,
precio_unit)
        VALUES (v_pedido_id, p_producto_ids[v_i], p_cantidades[v_i],
v_precio_unit)
        RETURNING linea_id INTO v_linea_id;

        UPDATE tienda.productos
           SET stock = stock - p_cantidades[v_i]
         WHERE producto_id = p_producto_ids[v_i];
```

```
        v_total_linea  := round(p_cantidades[v_i] * v_precio_unit, 2);
        v_total_pedido := v_total_pedido + v_total_linea;
        v_num_lineas   := v_num_lineas + 1;
    END LOOP;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE EXCEPTION 'Producto activo no encontrado en la línea %: id %',
            v_i, p_producto_ids[v_i];
END;
$$;
```

Cómo llamarlo

Desde SQL puro, con `CALL` y `NULL` en los `OUT`:

sql

```
CALL tienda.sp_crear_pedido_multilinea(
    1,                          -- cliente
    ARRAY[1, 3, 2],             -- productos
    ARRAY[2, 1, 4],             -- cantidades
    NULL, NULL, NULL            -- OUT
);
```

Devuelve una fila con `v_pedido_id`, `v_total_pedido`, `v_num_lineas`.

Desde otro bloque PL/pgSQL, con variables para recibir los `OUT`:

sql

```
DO $$
DECLARE
    r_pedido_id integer;
    r_total     numeric(12,2);
    r_lineas    integer;
BEGIN
    CALL tienda.sp_crear_pedido_multilinea(
        1, ARRAY[1,3,2], ARRAY[2,1,4],
        r_pedido_id, r_total, r_lineas
    );
    RAISE NOTICE 'Pedido % creado con % líneas, total %',
        r_pedido_id, r_lineas, r_total;
END;
$$;
```

Puntos clave del diseño

1. Un solo `INSERT` en `pedidos`

La cabecera se crea **antes** del bucle, una sola vez. Todas las líneas comparten el mismo `v_pedido_id`. Eso es lo que pedías: "varias líneas en un mismo pedido".

2. Todo o nada

El bloque `EXCEPTION` captura cualquier fallo (stock insuficiente, producto inactivo, cantidad inválida). Al lanzar `RAISE EXCEPTION`, **toda la transacción del procedimiento se revierte**,

incluido el `INSERT` de `pedidos` y los `UPDATE` de stock ya hechos en líneas anteriores. No queda pedido huérfano ni stock descontado a medias.

Esto funciona así porque el procedimiento **no hace `COMMIT` intermedio**. Si lo hicieras, perderías la atomicidad. Por eso aquí **no** se usa `COMMIT` dentro del bucle.

3. `FOR UPDATE` dentro del bucle

Bloquea cada producto **en el momento de procesar su línea**. Si dos sesiones intentan vender el mismo producto a la vez, la segunda espera a que la primera libere el lock (al terminar su transacción) y entonces ve el stock ya descontado. Evita sobreventa.

Orden de bloqueo: si dos pedidos incluyen los mismos productos en **orden distinto** (`[1,3]` vs `[3,1]`), puede haber **deadlock**. Si te preocupa, ordena los arrays por `producto_id` antes de recorrerlos. Para un sistema pequeño no suele ser problema, pero conviene saberlo.

4. Validación de longitudes

`array_length(..., 1)` devuelve `NULL` para arrays vacíos, por eso se comprueba con `IS NULL`. Y se exige que ambos arrays tengan la misma longitud; si no, el `FOR` accedería a índices fuera de rango.

5. `v_i` en el `EXCEPTION`

En el bloque `EXCEPTION` se usa `v_i` y `p_producto_ids[v_i]` para reportar **qué línea** falló. `v_i` conserva su valor porque el `EXCEPTION` está en el mismo bloque que el bucle.

Alternativa con tipo compuesto (más limpia si puedes crear tipos)

Si prefieres pasar las líneas como **un solo array de pares** en lugar de dos arrays paralelos:

sql

```
CREATE TYPE tienda.linea_input AS (
    producto_id integer,
    cantidad    integer
);
```

Y el procedimiento recibiría `p_lineas tienda.linea_input[]`, recorriéndolo con:

sql

```
FOR v_i IN 1 .. array_length(p_lineas, 1) LOOP
    -- p_lineas[v_i].producto_id, p_lineas[v_i].cantidad
END LOOP;
```

Se llama así:

sql

```
CALL tienda.sp_crear_pedido_multilinea(
    1,
    ARRAY[ROW(1,2), ROW(3,1), ROW(5,4)]::tienda.linea_input[],
    NULL, NULL, NULL
```

```
);
```
Es más legible cuando hay muchos campos por línea (producto, cantidad, descuento, etc.), pero requiere crear el tipo. Los dos arrays paralelos funcionan sin tocar el esquema.

Resumen

•**Un `INSERT` en `pedidos`** fuera del bucle → todas las líneas van al mismo pedido.

•**Bucle sobre arrays paralelos** (`p_producto_ids`, `p_cantidades`) → N líneas.

•**`FOR UPDATE` por producto** → sin sobreventa concurrente.

•**`EXCEPTION` sin `COMMIT` intermedio** → atomicidad total: si una línea falla, no queda ni pedido ni stock a medias.

•**`OUT`** devuelve `pedido_id`, total y número de líneas al llamador.